

Lab Assignment # 08

Course Title : **AI Assistant Coding**
Name of Student : **R Govardhan Sai Ganesh**
Enrollment No. : **2303A54044**
Batch No. : **48**

Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

• **Task:** Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

• **Requirements:**

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1: • Password validation logic passing all AI-generated test cases.

Output Screenshot:



```
lab 8 > lab 8.1.py > ...
1  #Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases
2  #Task Description #1 (Password Strength Validator – Apply AI in Security Context)
3  #• Task: Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.
4  #• Requirements: o Password must have at least 8 characters. o Must include uppercase, lowercase, digit, and special character. o Must not contain spaces.
5  #Example Assert Test Cases: assert is_strong_password("Abcd@123") == True assert is_strong_password("abcd123") == False assert is_strong_password("ABCD@1234") == True
6  #Expected Output #1: Password validation logic passing all AI-generated test cases.
7
8  import re
9
10 def is_strong_password(password: str) -> bool:
11     # Must have at least 8 characters
12     if len(password) < 8:
13         return False
14
15     # Must not contain spaces
16     if " " in password:
17         return False
18
19     # Must include uppercase, lowercase, digit, and special character
20     if not re.search(r"[A-Z]", password): # Uppercase
21         return False
22     if not re.search(r"[a-z]", password): # Lowercase
23         return False
24     if not re.search(r"[0-9]", password): # Digit
25         return False
26     if not re.search(r"[^A-Za-z0-9]", password): # Special character
27         return False
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

Python + v [Icons]

PS C:\Users\rgsga\OneDrive\Desktop\AI Assistant> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/AI Assistant/1.py"

All test cases passed ✓

Explanation: This checks if a password is truly “strong” by making sure it has the right mix of characters and no spaces.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

• Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.

- **Requirements:**

- o Classify numbers as Positive, Negative, or Zero.
- o Handle invalid inputs like strings and None.
- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
```

Expected Output #2: • Classification logic passing all assert tests.

Output Screenshot:

lab 8.1.py

lab 8 > lab 8.1.py > ...

```
51
52 #Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)
53 #• Task: Use AI to generate at least 3 assert test cases for a classify_number(n) function. Implement using loops.
54 #• Requirements: o Classify numbers as Positive, Negative, or Zero. o Handle invalid inputs like strings and None. o Include boundary conditions
55 #Example Assert Test Cases: assert classify_number(10) == "Positive" assert classify_number(-5) == "Negative" assert classify_number(0) == "Zero"
56 #Expected Output #2: • Classification logic passing all assert tests.
57
58 def classify_number(n):
59     # Handle invalid inputs
60     if n is None or isinstance(n, str):
61         return "Invalid Input"
62
63     # Using loop to classify (though simple if-else works, requirement says use loops)
64     for value in [n]:
65         if value > 0:
66             return "Positive"
67         elif value < 0:
68             return "Negative"
69         else:
70             return "Zero"
71
72 # Standard cases
73 assert classify_number(10) == "Positive" # Positive number
74 assert classify_number(-5) == "Negative" # Negative number
75 assert classify_number(0) == "Zero" # Zero case
76
77 # Boundary conditions
78 assert classify_number(1) == "Positive" # Smallest positive boundary
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

Python + [] [] []

PS C:\Users\r\rgsga\OneDrive\Desktop\AI Assistant> & C:/Users/r\rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/r\rgsga/OneDrive/Desktop/AI Assistant/.1.py"

All test cases passed ✓

Explanation: This sorts numbers into positive, negative, or zero, while politely rejecting anything that isn't a number.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- **Task:** Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

- **Requirements:**

- o Ignore case, spaces, and punctuation.
- o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
```

Expected Output #3: • Function correctly identifying anagrams and passing all AI-generated tests.

Output Screenshot:



```
lab 8.1.py x
lab 8 > lab 8.1.py > ...
87 #Task Description #3 (Anagram Checker - Apply AI for String Analysis)
88 #• Task: Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.
89 #• Requirements: o Ignore case, spaces, and punctuation. o Handle edge cases (empty strings, identical words).
90 #Example Assert Test Cases: assert is_anagram("listen", "silent") == True assert is_anagram("hello", "world") == False assert is_anagram("Dormi
91 #Expected Output #3: • Function correctly identifying anagrams and passing all AI-generated tests.
92
93 import string
94 def is_anagram(str1: str, str2: str) -> bool:
95     # Handle edge cases: empty strings
96     if not str1 or not str2:
97         return False
98
99     # Normalize: lowercase, remove spaces and punctuation
100     translator = str.maketrans('', '', string.punctuation + " ")
101     clean_str1 = str1.lower().translate(translator)
102     clean_str2 = str2.lower().translate(translator)
103
104     # Compare sorted characters
105     return sorted(clean_str1) == sorted(clean_str2)
106
107 # Standard cases
108 assert is_anagram("listen", "silent") == True # Classic anagram
109 assert is_anagram("hello", "world") == False # Not an anagram
110 assert is_anagram("Dormitory", "Dirty Room") == True # Ignore case & spaces
111
112 # Edge cases
113 assert is_anagram("", "") == False # Empty strings
114 assert is_anagram("Test", "Test") == True # Identical words
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + -

```
PS C:\Users\rgsga\OneDrive\Desktop\AI Assistant> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/AI Assistant
.1.py"
All test cases passed ✓
```

Explanation: This spots whether two words or phrases are made of the same letters, ignoring case, spaces, and punctuation.

Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)

- **Task:** Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

- **Methods:**

- o add_item(name, quantity)
- o remove_item(name, quantity)
- o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```

inv.add_item("Pen", 10)
assert inv.get_stock("Pen") == 10
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3

```

Expected Output #4: • Fully functional class passing all assertions.

Output Screenshot:

```

lab 8 > lab 8.1.py X
lab 8 > lab 8.1.py > ...
121 #Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)
122 #• Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
123 #• Methods: o add_item(name, quantity) o remove_item(name, quantity) o get_stock(name)
124 #Example Assert Test Cases:
125 """inv = Inventory()
126 inv.add_item("Pen", 10)
127 assert inv.get_stock("Pen") == 10
128 inv.remove_item("Pen", 5)
129 assert inv.get_stock("Pen") == 5
130 inv.add_item("Book", 3)
131 assert inv.get_stock("Book") == 3"""
132 #Expected Output #4: • Fully functional class passing all assertions.
133
134
135 class Inventory:
136     def __init__(self):
137         # Dictionary to store items and their quantities
138         self.stock = {}
139
140     def add_item(self, name: str, quantity: int):
141         if quantity < 0:
142             raise ValueError("Quantity cannot be negative")
143         # Add new item or update existing stock
144         if name in self.stock:
145             self.stock[name] += quantity
146         else:
147             self.stock[name] = quantity

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + v [] []

PS C:\Users\rgsga\OneDrive\Desktop\AI Assistant> & C:/Users/rgsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rgsga/OneDrive/Desktop/AI Assistan
 .1.py"
 All test cases passed ✓

Explanation: This simulates a mini store system where you can add, remove, and check stock for items.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

• **Task:** Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

• **Requirements:**

- o Validate "MM/DD/YYYY" format.
- o Handle invalid dates.
- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```

assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"

```

Expected Output #5: • Function passes all AI-generated assertions and handles edge cases.

Output Screenshot:



```
lab 8.1.py X
lab 8 > lab 8.1.py > ...

190 #Task Description #5 (Date Validation & Formatting - Apply AI for Data Validation)
191 #• Task: Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.
192 #• Requirements: o Validate "MM/DD/YYYY" format. o Handle invalid dates. o Convert valid dates to "YYYY-MM-DD".
193 #Example Assert Test Cases: assert validate_and_format_date("10/15/2023") == "2023-10-15" assert validate_and_format_date("02/30/2023") == "Invalid Date"
194 #Expected Output #5: • Function passes all AI-generated assertions and handles edge cases.
195
196
197 from datetime import datetime
198 def validate_and_format_date(date_str: str) -> str:
199     try:
200         # Try parsing with strict MM/DD/YYYY format
201         parsed_date = datetime.strptime(date_str, "%m/%d/%Y")
202         # Return formatted date as YYYY-MM-DD
203         return parsed_date.strftime("%Y-%m-%d")
204     except ValueError:
205         # If parsing fails, it's an invalid date
206         return "Invalid Date"
207
208 # Valid dates
209 assert validate_and_format_date("10/15/2023") == "2023-10-15" # Standard valid date
210 assert validate_and_format_date("01/01/2024") == "2024-01-01" # New Year boundary
211 assert validate_and_format_date("12/31/2025") == "2025-12-31" # End of year boundary
212
213 # Invalid dates
214 assert validate_and_format_date("02/30/2023") == "Invalid Date" # February 30 doesn't exist
215 assert validate_and_format_date("13/01/2023") == "Invalid Date" # Invalid month
216 assert validate_and_format_date("00/10/2023") == "Invalid Date" # Invalid month zero
217 assert validate_and_format_date("11/31/2023") == "Invalid Date" # November has only 30 days
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE Python + v [] []

PS C:\Users\rsga\OneDrive\Desktop\AI Assistant> & C:/Users/rsga/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/rsga/OneDrive/Desktop/AI Assistant/.1.py"

All test cases passed ✓

Explanation: This ensures a date is valid in “MM/DD/YYYY” format and neatly converts it into “YYYY-MM-DD”.